

PolarOps — What's Built, Why, and the Exact Backend Contract

This is a complete reference for everyone joining this project from here — a teammate picking up backend work, or an AI helping them. It covers what PolarOps is, every feature and why it exists, the full data model, and the exact API contract the frontend already codes against.

 **Read this first if you're the one doing backend work**

A working backend already exists — Express + SQLite, fully wired to every screen in the app. Nothing in the frontend is stubbed, mocked, or waiting on you to build from scratch.

Your job is one of two things: **(a)** harden this backend for real deployment (see the checklist near the bottom), or **(b)** replace it with a different stack while honoring the exact same API contract documented here — either way, the frontend does not need to change if the contract holds.

Jump straight to [Backend wiring checklist](#) if you just want the action items.

What PolarOps is

PolarOps is a duty-officer command dashboard for polar research logistics — modeled on the kind of operations screen an organization like NCPOR would use to track expeditions, cargo, supplies, personnel, and emergencies across Antarctic and Arctic stations. It's built as a real full-stack app: a React frontend talking to a real Express API backed by a real SQLite database, with actual authentication, actual role permissions, and actual computed statistics — not a static mockup.

The visual language is deliberately "glacier / frost" — icy blues, glass-panel cards, a deep navy text color instead of black — chosen specifically to read as "polar operations" rather than a generic SaaS admin template. That design system is locked into `tailwind.config.js` as tokens; nothing in the app uses raw hex colors outside of Leaflet/Recharts styling, which can't consume Tailwind classes and are documented inline wherever that exception applies.

Tech stack

LAYER	CHOICE	NOTES
Frontend framework	React 19 + Vite	SPA, no SSR
Routing	react-router-dom v7	Real URLs, protected routes, redirect-to-intended-page after login
Styling	Tailwind CSS v3	Design tokens in <code>tailwind.config.js</code> ; no CSS-in-JS
Icons	lucide-react	Outline style, stroke-width 1.75 throughout
Maps	react-leaflet + OpenStreetMap tiles	Real station coordinates, real expedition route polylines
Charts	recharts	All charts render real computed data — nothing fabricated
Testing	vitest + @testing-library/react	39 tests, pure-logic + component coverage
PWA	vite-plugin-pwa	Installable, service worker precaches the app shell; API calls always go live (never cached)
Backend framework	Express (Node, ESM)	Runs on port 4001
Database	better-sqlite3	Single file at <code>server/polarops.sqlite</code> , synchronous driver
Auth	jsonwebtoken + bcryptjs	Real password hashing, real signed JWTs, 12h expiry
AI	Google Gemini (gemini-3.8-flash)	Called server-side only — the API key never reaches the browser

Running it today

Two processes, started together with one command from the project root:

```
npm install          # root — installs frontend deps
npm install --prefix server # backend deps (only needed once)
npm run dev          # starts BOTH: Vite on :5183 and Express on :4001
```

Frontend: `http://localhost:5183` · Backend: `http://localhost:4001/api`

Demo accounts (seeded automatically on first run)

EMAIL	PASSWORD	ROLE
duty.officer@polarops.io	icebreaker26	duty_officer — full read/write, no delete
admin@polarops.io	glacieradmin26	admin — everything, including delete

The database auto-seeds realistic starter data (10 real polar research stations, sample expeditions, cargo, inventory, personnel, emergencies) the first time it runs — see [schema section](#) for the full seed logic.

Architecture map

Frontend (`src/`)

```
src/  
  pages/           one file per screen/route  
  components/  
    layout/       Sidebar, Topbar, AppLayout,  
                  MobileDrawer, ProtectedRoute  
    ui/           generic reusable pieces:  
                  DataTable, Modal, Pagination,  
                  StatusBadge, Reveal, Toast, etc.  
    map/          PolarMap (Leaflet)  
    emergency/    CompoundRiskCard + modal  
    expeditions/  cargo/ inventory/ personnel/  
                  — resource-specific modals & charts  
    analytics/    the 3 custom analytics charts  
    context/      AuthContext, ToastContext  
    hooks/        useApi, useListState,  
                  useKeyboardShortcuts, useScrollReveal  
    lib/          api.js (HTTP client + Blackout Mode),  
                  compoundRisk.js (pure correlation logic),  
                  format.js  
    data/         navigation.js, modules.js, stats.js  
                  (UI config, not app data)
```

Backend (`server/`)

```
server/  
  index.js        Express app, mounts every route  
  db.js           schema + migrations + seed data  
  .env            secrets (gitignored)
```

```
.env.example    template for the above
lib/
  auth.js       JWT sign/verify, role middleware
  query.js      shared pagination/sort helper
  audit.js      writes to audit_log table
  gemini.js     Gemini HTTP client w/ retry
routes/
  auth.js, expeditions.js, cargo.js,
  inventory.js, personnel.js, emergencies.js,
  stations.js, stats.js, auditLog.js,
  analytics.js, assistant.js
```

Every route file follows the same shape: `GET /` (paginated list), `GET /:id` (single record), `POST /` (create), `PATCH /:id` (update), `DELETE /:id` (admin-only). The exact contract for each is in the [API reference](#) below.

Feature inventory — what's built, and why

Every panel below is real and wired end-to-end. Each entry explains what it is, why it exists, and exactly which files and endpoints it depends on — so nothing here is a mystery to whoever (human or AI) reads this next.

Authentication & role-based access REAL

What: Email/password login, JWT session (12h expiry), two roles (`duty_officer` , `admin`). Login redirects back to whatever page the user originally tried to reach.

Why: A duty-officer console needs real access control — not everyone should be able to delete records. This also demonstrates the permission model any real deployment needs.

FILES

- `src/context/AuthContext.jsx` — holds token/user, exposes `login()` / `logout()` / `isAdmin`
- `src/pages/Login.jsx` , `src/components/layout/ProtectedRoute.jsx`
- `server/lib/auth.js` — `signToken` , `requireAuth` , `requireRole`

BACKEND DEPENDENCY

`POST /api/auth/login` , `GET /api/auth/me` — see [full contract](#).

Dashboard REAL

What: Landing page — 5 live stat tiles, the station map, a merged "Live Operational Feed" (low-stock + open emergencies sorted by recency), and a 6-tile quick-access grid.

Why: The single-glance overview a duty officer checks first. Every number is a real SQL aggregate, not a placeholder — this was explicitly built to avoid "fake dashboard syndrome."

BACKEND DEPENDENCY

`GET /api/stats` (all 5 tiles), `GET /api/emergencies?status=Open` + `GET /api/inventory?lowStock=true` (the feed).

Station Map REAL

What: Interactive Leaflet map with 10 real Antarctic/Arctic research stations (actual names and coordinates — Amundsen-Scott, McMurdo, Vostok, Ny-Ålesund, etc.), plus polylines showing active/planned/completed expedition routes drawn from real waypoint data. Click a station in the side list to fly to it; a "Reset view" button re-fits all stations; a live distance scale bar.

Why: Two real bugs were found and fixed while building this — worth knowing about if you touch it. First, the initial tile provider (CARTO) started requiring an API key mid-build; swapped to plain OpenStreetMap tiles with a CSS filter for the icy look. Second, the initial map view put 8 of 10 stations completely outside the visible viewport (a wide-short container doesn't suit pole-to-pole data in Mercator projection) — fixed by fitting the view to actual station bounds instead of a guessed center/zoom.

FILES

- `src/components/map/PolarMap.jsx` , `src/pages/MapPage.jsx`

BACKEND DEPENDENCY

`GET /api/stations` , `GET /api/expeditions` (for route polylines, uses each expedition's `waypoints` array).

Expeditions / Cargo / Inventory / Personnel REAL

What: Four full CRUD resource screens, each with: a stat strip, filter chips, live search, a sortable/paginated table, a "New ___" modal that actually creates records, a detail drill-down page (`/resource/:id`), and role-gated delete (admin only, enforced both in the UI and server-side with a 403).

Why: This is the operational core of the app — the actual data duty officers manage day to day. Inventory and Personnel detail pages also support inline editing.

Personnel check-in, specifically: added a real `last_checkin` timestamp column and a dedicated `POST /:id/checkin` endpoint, plus a "Check In" button on the Personnel detail page. This didn't exist originally — it was added specifically to make the Compound Risk feature's "overdue personnel" detection real instead of mocked (see below).

Inventory, specifically: originally had no per-station stock — it was a single global pool. Added a real `station_id` column because "low stock at a station" is meaningless without it, and Compound Risk needs to correlate by station.

FILES

- `src/pages/{Expeditions,Cargo,Inventory,Personnel}.jsx` + matching `*Detail.jsx`
- `src/components/{expeditions,cargo,inventory,personnel}/New*Modal.jsx`
- `src/components/ui/{DataTable,Pagination,StatStrip,StatusBadge,SearchInput,FilterChip}.jsx` — shared across all four

BACKEND DEPENDENCY

Full CRUD contract for each — see [API reference](#) sections below.

Emergency Response + Compound Risk correlation REAL

What: Standard incident timeline (report / resolve, with severity), plus a correlation feature called **Compound Risk**: when 2+ independent problem types (low stock, an overdue personnel check-in, an open emergency) hit the *same station* at the same time, a distinct elevated card appears above the regular list summarizing the correlation, with a severity rating and a one-shot pulse animation.

Why: Three unrelated alerts at three different stations are normal. Three unrelated alerts at *one* station is a pattern worth flagging — that's the actual value proposition of this feature.

Severity algorithm (implemented exactly, verified by 14 unit tests): CRITICAL if any co-occurring condition is an open emergency; HIGH if exactly 2 conditions co-occur and one is an overdue check-in (with no emergency present); MEDIUM otherwise. **Known finding, not a bug:** with only these 3 condition kinds, MEDIUM is mathematically unreachable — every possible pairing resolves to CRITICAL or HIGH. This is a documented property of the algorithm as specified, not a defect in the implementation.

FILES

- `src/lib/compoundRisk.js` — pure correlation logic, zero React/browser dependency, fully unit-tested (`compoundRisk.test.js`)
- `src/components/emergency/CompoundRiskCard.jsx` , `src/pages/Emergency.jsx`

BACKEND DEPENDENCY

No dedicated endpoint — computed client-side from data already fetched: `GET /api/inventory` , `GET /api/personnel?status=In Field` , `GET /api/emergencies?status=Open` .

Analytics REAL

What: 6 charts, all computed from real timestamped records — nothing is a fabricated time-series. Emergency reports and cargo shipments over time (grouped by real `created_at` / `reported_at` dates), response-time trend (real report→resolve durations), a custom Gantt-style expedition timeline, personnel deployment breakdown, and stock-by-category.

Why: Trend charts are only honest if the underlying events actually happened at those times — which they do here, since every record already carries a real timestamp. No synthetic history table was added; everything is derived from existing columns.

FILES

- `src/pages/Analytics.jsx` , `src/components/analytics/*.jsx` , `src/components/ui/TrendChart.jsx`

BACKEND DEPENDENCY

`GET /api/analytics` — one endpoint, all 6 datasets, computed with SQL date grouping. See [contract](#).

Audit Log REAL

What: Every create/update/delete across every resource (expeditions, cargo, inventory, personnel — including check-ins — and emergencies) is written to a dedicated `audit_log` table: who did it (from the real JWT, not client-supplied), what happened, and a human-readable summary. A filterable, sortable, searchable page shows the full history.

Why: Operational accountability — a duty-officer console handling emergencies and safety-relevant data should be able to answer "who changed this and when" without guessing.

FILES

- `server/lib/audit.js` — one `recordAudit()` call added at every write point across all 5 resource route files. `src/pages/AuditLog.jsx`.

BACKEND DEPENDENCY

`GET /api/audit-log` — see [contract](#).

Polar Blackout Mode LABELED SIMULATION

What: A demo-triggerable "simulated connectivity loss." Click the status pill (top-right, normally "● SYSTEM ONLINE") to flip into blackout: every write (POST/PATCH) from anywhere in the app queues in memory instead of hitting the API, while reads keep working live. Toggling back off replays the queue sequentially against the real backend, with a visible per-item sync animation and a failure/retry path if one call errors.

Why explicitly labeled "simulation," not "offline-first": the backend never actually goes down — only the frontend pretends writes can't go through. The queue is in-memory only and resets on page reload; there's no service worker or IndexedDB involved. This is intentional and stated in the UI itself (a permanent caption: "Simulated interruption — demonstrates queued sync. Persistent offline storage is a roadmap item.") so it never overclaims what it does.

How it avoided touching every screen: the interception happens in exactly one place — the shared request function in `src/lib/api.js`. Every screen that performs a write (`Cargo.jsx`, `PersonnelDetail.jsx`, etc.) calls the exact same `api.post()` / `api.patch()` it always did — none of them know Blackout Mode exists.

FILES

- `src/lib/api.js` — `toggleBlackout()`, `drainQueue()`, `retryQueueItem()`, `subscribeBlackout()`
- `src/components/layout/Topbar.jsx` (the toggle pill + live timer),
`src/components/ui/OfflineQueuePanel.jsx` (the floating queue panel)

BACKEND DEPENDENCY

None new — it replays against the exact same write endpoints documented below, just delayed.

AI Assistant REAL

What: A floating chat panel (bottom-right) that answers questions about current operations — "which stations have low stock?", "are any personnel overdue for check-in?" — grounded in real live data, not a canned or hallucinated response.

Why / how it stays honest: the backend gathers real current data (stats, low-stock items with station names, overdue personnel with real timestamps, open emergencies, expeditions, cargo) into a JSON context block, and instructs the model explicitly: answer *only* from this data, say so plainly if something isn't in it, never invent a station or number. Verified live — asked "what emergencies are open?" and got back the two real open incidents with correct stations, severities, and timestamps.

Known limitation: running on Gemini's free tier, capped at **5 requests per minute**. A 429 from that cap shows a clean "try again in about a minute" message rather than retrying pointlessly or crashing. A 503 ("high demand," observed as a real, currently-common condition on this model) gets short automatic retries instead.

FILES

- `server/lib/gemini.js` — raw HTTP client (no SDK — Gemini isn't Anthropic), retry-on-503, clean 429 message
- `server/routes/assistant.js` — builds the data context, owns the system prompt
- `src/components/ui/AssistantPanel.jsx`

BACKEND DEPENDENCY

`POST /api/assistant/chat` — see [contract](#). Requires `GEMINI_API_KEY` in `server/.env`.

Smaller UX features REAL

Command palette (`Ctrl/Cmd+K`) — fuzzy jump to any page.

`src/components/ui/CommandPalette.jsx` .

Keyboard shortcuts — `g` then a letter (`d` =dashboard, `m` =map, etc.) for quick nav.

`src/hooks/useKeyboardShortcuts.js` .

Settings page — real user profile from the JWT, persisted notification preferences (localStorage).

`src/pages/Settings.jsx` .

Toasts — global success/error feedback after every write action.

`src/context/ToastContext.jsx` .

PWA — installable, app shell precached; verified the service worker actually activates and the manifest is valid. API calls are explicitly excluded from caching (`NetworkOnly`) so operational data is never served stale.

Snowflake click effect — a small themed one-shot animation on click, purely decorative, respects

`prefers-reduced-motion` .

Data model / schema reference

SQLite, defined in `server/db.js` . Every `ALTER TABLE` migration is idempotent (checks `PRAGMA table_info` before adding a column), so it's safe to run against either a fresh or an existing

database file.

users

COLUMN	TYPE	NOTES
id	INTEGER PK	autoincrement
email	TEXT UNIQUE	
password_hash	TEXT	bcrypt, 10 rounds
name	TEXT	
role	TEXT	'duty_officer' 'admin'
created_at	TEXT	default now

stations

COLUMN	TYPE	NOTES
id	TEXT PK	slug, e.g. "mcmurdo"
name	TEXT	real station name
region	TEXT	'Antarctic' 'Arctic'
lat, lng	REAL	real coordinates

Seeded with 10 real stations: Amundsen-Scott, McMurdo, Palmer, Halley VI, Concordia, Vostok, Ny-Ålesund, Alert, Summit Camp, Barentsburg.

expeditions

COLUMN	TYPE	NOTES
id	INTEGER PK	
name	TEXT	
status	TEXT	'Planned' 'Active' 'Completed'
region	TEXT	free text (not FK'd to stations.region)
start_date	TEXT	required, YYYY-MM-DD
end_date	TEXT	nullable
team_lead	TEXT	
waypoints	TEXT	JSON-encoded array of station IDs, e.g. ["mcmurdo","vostok"] — parsed to a real array by the API layer before it reaches the client
created_at, updated_at	TEXT	

cargo

COLUMN	TYPE	NOTES
id	INTEGER PK	
manifest_id	TEXT	not unique-constrained — duplicates are possible today
description	TEXT	
status	TEXT	'Delivered' 'In Transit' 'Delayed' 'Pending'
origin, destination	TEXT	free text
weight_kg	REAL	default 0
created_at, updated_at	TEXT	

inventory

COLUMN	TYPE	NOTES
id	INTEGER PK	
name, category	TEXT	
station_id	TEXT	FK → stations.id, nullable. <i>Added mid-project</i> — see Compound Risk rationale above.
quantity, threshold	INTEGER	"low stock" = <code>quantity ≤ threshold</code>
unit	TEXT	default <code>'units'</code>
needs_maintenance	INTEGER	0/1 boolean
created_at, updated_at	TEXT	

personnel

COLUMN	TYPE	NOTES
id	INTEGER PK	
name, role	TEXT	
station_id	TEXT	FK → stations.id, nullable
status	TEXT	<code>'In Field' 'On Leave' 'Base'</code>
last_checkin	TEXT	nullable. <i>Added mid-project</i> for real overdue-detection. "Overdue" = In Field AND (never checked in, or >24h since last check-in).
created_at, updated_at	TEXT	

emergencies

COLUMN	TYPE	NOTES
id	INTEGER PK	
title	TEXT	
severity	TEXT	'critical' 'warning' 'resolved' — note lowercase, unlike other status fields
status	TEXT	'Open' 'Resolved'
station_id	TEXT	FK → stations.id, nullable
reported_at	TEXT	default now
resolved_at	TEXT	nullable; auto-set server-side the moment status transitions to 'Resolved'

audit_log

COLUMN	TYPE	NOTES
id	INTEGER PK	
user_id	INTEGER	FK → users.id
user_name	TEXT	denormalized at write time (from the JWT)
action	TEXT	'create' 'update' 'delete'
resource_type	TEXT	e.g. 'expedition', 'inventory'
resource_id	TEXT	
summary	TEXT	human-readable, e.g. Updated inventory item "Diesel fuel" (4200/5000 L)
created_at	TEXT	

API reference — the exact contract

Base URL is hardcoded today as `http://localhost:4001/api` in `src/lib/api.js` (the `API_BASE` constant) — that's the one line to change for a deployed URL. Every write endpoint requires `Authorization: Bearer <jwt>` unless marked `PUBLIC`. Admin-only endpoints are marked `ADMIN`. List endpoints share pagination params: `page`, `pageSize` (max 100), `sort`, `order` (`asc` / `desc`), returning `{ data, total, page, pageSize, totalPages }`.

Auth

POST /api/auth/login **PUBLIC**

Body: { email, password }

200: { token, user: { id, email, name, role } } · **401** on bad credentials

GET /api/auth/me

200: { user: { id, email, name, role } } — decoded straight from the JWT, no DB hit

Expeditions

GET /api/expeditions **PUBLIC**

Query: status, region, q (matches name/team_lead), + shared pagination

Row shape: all columns plus waypoints parsed to a real array

GET /api/expeditions/:id **PUBLIC**

200: { data } · **404** if not found

POST /api/expeditions

Body: { name, status, region, start_date, end_date?, team_lead, waypoints? } — first 5 required

201: { data } · writes an audit log entry

PATCH /api/expeditions/:id

Body: any subset of the same fields (merge-patch semantics)

DELETE /api/expeditions/:id **ADMIN**

204 on success

Cargo

GET /api/cargo **PUBLIC**

Query: status, q (description/manifest_id/origin/destination) + pagination

GET /api/cargo/:id **PUBLIC**

POST /api/cargo

Body: { manifest_id, description, status, origin, destination, weight_kg? }

PATCH /api/cargo/:id

DELETE /api/cargo/:id **ADMIN**

Inventory

GET /api/inventory **PUBLIC**

Query: category, lowStock=true (quantity ≤ threshold), needsMaintenance=true, q

Row shape: joined with stations → adds station_name, station_region

GET /api/inventory/:id **PUBLIC**

POST /api/inventory

Body: { name, category, station_id?, quantity?, threshold?, unit?, needs_maintenance? }

PATCH /api/inventory/:id

DELETE /api/inventory/:id **ADMIN**

Personnel

GET /api/personnel **PUBLIC**

Query: status, q · joined with stations → station_name, station_region

GET /api/personnel/:id **PUBLIC**

POST /api/personnel

Body: { name, role, station_id?, status }

PATCH /api/personnel/:id

POST /api/personnel/:id/checkin

No body. Sets last_checkin = now(). This is the endpoint Blackout Mode's "check-in" demo action queues.

DELETE /api/personnel/:id **ADMIN**

Emergencies

GET /api/emergencies **PUBLIC**

Query: status, q (title) · joined with stations → station_name

Note: there is intentionally no GET /:id — the frontend has no emergency detail page, only the list.

POST /api/emergencies

Body: { title, severity, station_id? } — status is always forced to 'Open' server-side

PATCH /api/emergencies/:id

Setting status: "Resolved" auto-stamps resolved_at server-side if it wasn't already resolved.

DELETE `/api/emergencies/:id` **ADMIN**

Stations, Stats, Audit Log, Analytics, Assistant

GET `/api/stations` **PUBLIC**

200: `{ data: [{ id, name, region, lat, lng }] }` — all 10, no pagination

GET `/api/stats` **PUBLIC**

200: `{ dashboard, cargo, inventory, personnel, emergency }` — 5 objects, each a set of live COUNT/AVG aggregates. Exact keys are read directly in `src/data/stats.js` and each resource page — grep for `statKey` if renaming any of them.

GET `/api/audit-log`

Query: `resourceType`, `action`, `q` + pagination, default sort `created_at desc`

GET `/api/analytics`

200: `{ emergenciesByDay, responseTimeTrend, cargoByDay, expeditionTimeline, personnelBreakdown, inventoryByCategory }` — no query params, computes everything server-side with SQL date grouping

POST `/api/assistant/chat`

Body: `{ message }`

200: `{ reply }` · **502** with `{ error }` if Gemini fails or quota is hit

Requires `GEMINI_API_KEY` set in `server/.env` — see [env vars](#).

Backend wiring checklist

Concrete, ordered action items — whether you keep this Express/SQLite backend or replace it with something else. Priority reflects "will break something real" vs. "nice to have before shipping."

- ☐ **HIGH** Set real secrets in `server/.env` before any non-local deployment: `JWT_SECRET` (a long random string — currently falls back to a known dev value if unset, which lets anyone

forge an admin token) and `GEMINI_API_KEY`. Template is in `server/.env.example`. Never commit `.env` — already gitignored.

- ☐ **HIGH** **Decide: keep SQLite or migrate to a real database.** `better-sqlite3` works fine for a demo/single-instance deployment but doesn't scale to multiple server instances (one file, one process). If moving to Postgres/MySQL, the schema in [the schema section](#) translates almost directly — the only SQLite-specific things are `datetime('now')` defaults and the `julianday()` date-math used in `stats.js` / `analytics.js` / `assistant.js` for overdue-check-in and response-time calculations, which need Postgres equivalents (`NOW()` , `EXTRACT(EPOCH FROM ...)`).
- ☐ **HIGH** **Update `API_BASE` in `src/lib/api.js`** to the deployed backend URL (currently hardcoded to `localhost:4001`). Ideally make it a Vite env var (`import.meta.env.VITE_API_BASE`) so dev/staging/prod can differ without a code change.
- ☐ **HIGH** **CORS is currently wide open** (`app.use(cors())`) with no options in `server/index.js`). Restrict to the actual frontend origin before deploying anywhere public.
- ☐ **MEDIUM** **Rate limiting** — there is none on the Express side today (only Gemini's own free-tier cap protects the assistant route). Add something like `express-rate-limit` on `/api/auth/login` at minimum before this is public-facing.
- ☐ **MEDIUM** **Gemini quota** — free tier is 5 requests/minute, fine for internal demo use, not for real multi-user traffic. Either upgrade the Gemini plan or swap the assistant to a different provider (Claude Haiku is a good low-cost option) — only `server/lib/gemini.js` and `server/routes/assistant.js` would need to change; the frontend contract (`POST /api/assistant/chat`) can stay identical.
- ☐ **MEDIUM** **No unique constraint on `cargo.manifest_id`** — duplicates are currently possible. Add a `UNIQUE` constraint if manifest IDs need to be truly unique in production.
- ☐ **LOW** **Bundle size** — the frontend production bundle is ~930KB minified (Leaflet + Recharts + Router pull real weight). Not a backend concern, but worth code-splitting with `React.lazy()` per-route if load time matters.
- ☒ **DONE** Password hashing (bcrypt), JWT auth, role-based delete restriction, and audit logging are already real and don't need rework — just real secrets (above).

Environment variables

All backend-only, in `server/.env` (gitignored; `server/.env.example` is the checked-in template). None of these should ever be exposed to the frontend — a `VITE_`-prefixed variable gets bundled straight into the public JS.

VARIABLE	REQUIRED	PURPOSE
<code>JWT_SECRET</code>	Yes, for real deployment	Signs/verifies auth tokens. Falls back to a fixed dev string if unset — do not ship that fallback.
<code>GEMINI_API_KEY</code>	Yes, for the AI Assistant to work	Server-side only, read in <code>server/lib/gemini.js</code> . Get one at aistudio.google.com .
<code>PORT</code>	No	Defaults to <code>4001</code> .

Gotchas & known limitations

- ☐ **NOTE** **Blackout Mode's queue is genuinely in-memory only** — a page refresh mid-blackout loses anything queued. This is intentional and disclosed in the UI, not an oversight.
- ☐ **NOTE** **Compound Risk has no dedicated backend endpoint** — it's computed entirely client-side from data the Emergency page already fetches. If you build a "real" correlation engine server-side later, `src/lib/compoundRisk.js` is the exact spec to match (and its 14 tests are the exact behavior to preserve).
- ☐ **NOTE** **Emergency severity is lowercase** (`'critical'/'warning'/'resolved'`) while every other status field in the schema is capitalized (`'Open'` , `'In Field'` , etc.) — inconsistent, but every frontend call site already matches it exactly. Don't silently "fix" the casing without updating every reference.
- ☐ **NOTE** **Analytics date grouping uses SQLite's `date()` function** directly in `server/routes/analytics.js` — a straightforward swap for whatever the equivalent is in a different database engine.
- ☐ **NOTE** **The seed data intentionally uses real place names** (actual polar research stations, real coordinates) but fictional operational data (expeditions, personnel, incidents are all made up). This distinction was a deliberate choice throughout the project — geography is real, operations are demo data.

PolarOps — written for whoever picks up backend work next, human or AI. If anything here goes stale as the code changes, this document should be updated alongside it, not treated as separately-maintained documentation.